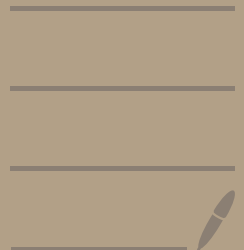


JAX Notebook

w/ Math & Code
explanations for beginners



```
import matplotlib.pyplot as plt

x_jnp = jnp.linspace(0, 10, 1000)
y_jnp = 2 * jnp.sin(x_jnp) * jnp.cos(x_jnp)
plt.plot(x_jnp, y_jnp);
```

import the library as an **alias** (a nickname for the library)
 # jnp.linspace (starting-point, end-point, evenly spaces)
 # compute $y = 2\sin x \cos x$ for each x value
 $\approx \sin 2x$
 # .plot(x, y) draws the curve of y versus x

From 0 to 10, with 1000 evenly spaces

$$\frac{10}{1000} = 0.01$$

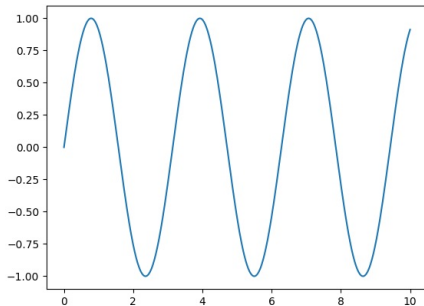
$\therefore 0, 0.01, 0.02, 0.03, \dots, 9.99, 10$

$$y = 2\sin x \cos x \\ = \sin 2x$$

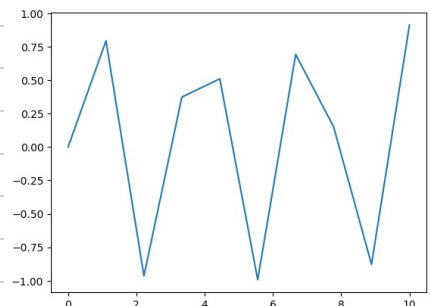
Substitute an example value of x for example 0.01

$$y = \sin 2(0.01) \\ \approx 3.99 \times 10^{-4} \\ \approx 0.000399$$

Since the distance between two x values is just 0.01, the graph will be smooth like butter, we can tell the difference once the spacing is reduced.



$\sin 2x \quad 0, 10, 1000$



$\sin 2x \quad 0, 10, 10$

The reason behind why $\sin 2x$ generates twice oscillating curves when compared to $\sin x$ even though abstractedly, they're still $\sin \theta$ is because of...

$\sin \theta$ oscillates from -1 to 1 and 360° (or 2π) makes up a full cycle around the loop. which means that for an argument x to travel a distance of 10 as the total distance and each time it travels 2π or 6.28 it makes a cycle, then...

$$\text{total number of cycles} = \frac{\text{total distance}}{\text{one cycle}} = \frac{10}{6.28} = 1.59 \quad \sin x$$

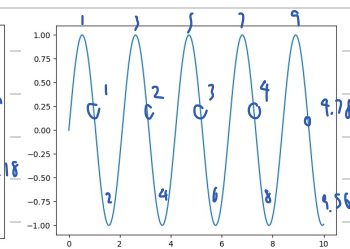
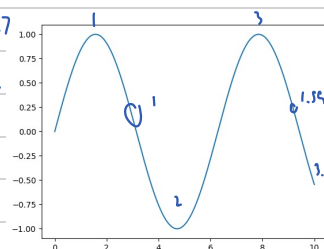
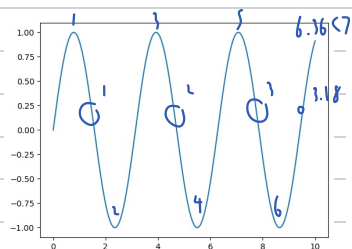
$$\sin 2x = \frac{2(10)}{6.28} = 3.18 \text{ cycles}$$

Because $x, c \rightarrow 10 \therefore 2x \quad 2(c) \rightarrow 2(10) \therefore 0 \rightarrow 20$
 $\therefore$ Larger distance requires more cycles to complete $\therefore$ More oscillations

Also $2(\text{one cycle}) = \text{extremes} \therefore 2(3.18) = 6.36 \text{ extremes } \sin 2x$

$2(1.59) = 3.18 \text{ extremes } \sin x$

$2(4.78) = 9.56 \text{ extremes } \sin 3x$



Just-in-Time Compilation with jax.jit

At default setting, JAX executes operations sequentially, with `JIT`, sequences of operations can be optimised together to run at once.

However, not all JAX code can be optimized, as it requires the arrays to be static and known at compile time

All JAX operations are expressed in XLA and we can compile this sequence of operations with the `jax.jit` function and XLA compiler.

- Open source compiler for machine learning
- Optimizes ML models in JAX to run faster and use less memory

```
installation.ipynb

import jax.numpy as jnp

def norm(X):
    X = X - X.mean(0)
    return X / X.std(0)
```

A function that normalizes the rows of a 2D matrix

Before diving into the function itself, we must understand how a matrix works.

All matrices are inherently 2D but in JAX or other libraries, they're referring to a 2D array with rows and columns, with each row having a vector.

Basically this is a matrix

[3, 4]

[6, 8]

It has two rows two values so the shape is (2,2)

A ^(row) vector has direction and magnitude
and in Matrices this is how it looks like.

Direction is the ratio between vector values

e.g. $\frac{3}{4}$

notice how row1 and row2 have the same direction is because they are scaled
replacers of one another.

for a 2D matrix (in code) with 3D vectors

shape: (2, 3)

[3, 6, 9]

[2, 1, 4]

the ratio (direction) becomes 1:2:3 for row1 and 2:1:4 for row2

length is the magnitude of a vector and it's computed by (pyth. theorem)

e.g. shape (2, 3)

[2, 4, 6] $\sqrt{2^2 + 4^2 + 6^2} = \sqrt{56} = 7.48$

[1, 3, 7] $\sqrt{1^2 + 3^2 + 7^2} = \sqrt{59} = 7.68$

Normalization

When we want to compare two vectors at different scales, we normalize them into
unit-length vectors to ensure fair comparison

↳ length: 1

Method: Divide each vector value by it's length, square them and sum it, then square root.

e.g. $\sqrt{\left(\frac{2}{\sqrt{56}}\right)^2 + \left(\frac{4}{\sqrt{56}}\right)^2 + \left(\frac{6}{\sqrt{56}}\right)^2} = 1$ while component values can be ≥ 1 , normalization
makes their length = 1

Original row vs Normalized row

[2, 4, 6] $\left[\frac{2}{\sqrt{56}}, \frac{4}{\sqrt{56}}, \frac{6}{\sqrt{56}}\right]$

Note that if all values are positive then after normalization they are in the range
of 0 to 1 but if negative could have negative normalized values. However most importantly
their magnitude (length) = 1 after normalization.

```
installation.ipynb

1 # Set the seed
2 np.random.seed(42)
3
4 # This will produce the same output every time the script is run
5 print(np.random.rand(3))
```

Numpy uses a seed as starting point to ensure same results are generated, if no seed is provided, Numpy uses your system time which every run generates different results.

`np.random.rand(3)` generates an array of size 3 with each value being a float.
[0.37454012, 0.95071431, 0.73199394]

Converting the numpy array into JAX arrays will get the same values under atol (absolute tolerance) which means ignoring minor numeric differences

★ However, if you run the `np.random.rand(3)` twice and printing them out, you'll realise the values are different, so you call it a couple times and get diff values. But if you reload the script, the values start from the beginning again.

JAX arrays are immutable and Numpy arrays are mutable

```
installation.ipynb

1 import numpy as np
2 import jax.numpy as jnp
3
4 np.random.seed(1701)
5 print(np.random.rand(10000, 10))
6 print(np.random.rand(10000, 10))
```

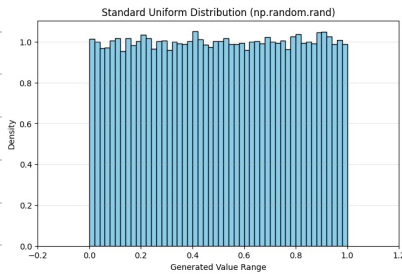
This will generate a 10000 row x 10 column array of floats

```
installation.ipynb

1 import jax.numpy as jnp
2 from jax import jit
3 import numpy as np
4
5 def norm(X):
6     X = X - X.mean(0)
7     return X / X.std(0)
8
9 norm_compiled = jit(norm)
10
11 np.random.seed(1701)
12 X = jnp.array(np.random.rand(10000, 10))
13 np.allclose(norm(X), norm_compiled(X), atol=1E-6)
```

`norm_compiled = jit(norm)` use JIT and XLA compilation to execute a sequence of operations at once

`np.allclose` compares all values of an array under absolute tolerance = $1E-6$ ignoring minor numeric differences and returns a boolean value. This proves JAX arrays = numpy arrays and JIT compiled operations = normal operations



```

1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # 1. Generate 100,000 samples between 0 and 1
5 data_uniform = np.random.rand(100000)
6
7 # 2. Plot the histogram
8 plt.figure(figsize=(8, 5))
9 plt.hist(data_uniform, bins=50, density=True, color="skyblue", edgecolor="black")
10
11 # 3. Add labels and formatting
12 plt.title("Standard Uniform Distribution (np.random.rand)")
13 plt.xlabel("Generated Value Range")
14 plt.ylabel("Density")
15 plt.xlim(-0.2, 1.2) # Extended slightly past 0 and 1 to show the strict boundaries
16 plt.grid(axis="y", alpha=0.3)
17
18 # 4. Display the plot
19 plt.show()

```

`numpy.random.rand()`

- uses standard uniform distribution
ie equal chance for each number to be selected
- range: $0.0 \leq x < 1.0$

example

- 100000 samples randomly selected in the range
- 50 bars * close values grouped together
- density=true * rough scaled down version of amount of times a value has appeared

shape analysis

- ∴ equal chance
- ∴ roughly appeared the same amount of times ie y-axis
- ∴ rough rectangle

fluctuation (some values appeared more times)

Math behind the plot ★ The Gold Standard Rule

the rule: area must be 1.0 aka 100% of the data

how and why?

$$\text{Area} = W \times H$$

$$W = 1.0 = 1$$

Divide the width into 50 equal sections ie width for each bar

$$1 \div 50 = 0.02$$

∴ Matplotlib scales on **density**, heights perfectly balance out each other e.g. one bar height: 1.05 the other one goes: 0.95

$$\therefore \frac{1.05 + 0.95}{2} = 1$$

$$\text{avg} = \frac{\text{Sum of height}}{\text{total bars}} = \frac{\text{bar}_1 + \text{bar}_2 + \text{bar}_3 + \dots + \text{bar}_{50}}{50} = 1.0$$

let's stack up all bars into one enormous bar

$$\therefore W = 0.02$$

$$\therefore H = \text{Sum of height} = 50$$

$$\therefore 0.02 \times 50 = 1.0$$

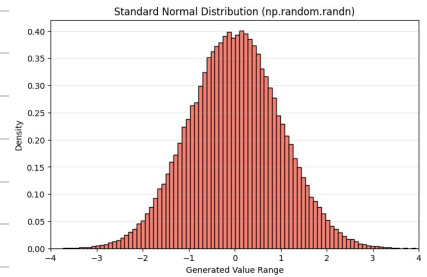
∴ total area is 1.0

∴ 100% of the data is represented on graph

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # 1. Generate 100,000 samples centered around 0
5 data_normal = np.random.randn(100000)
6
7 # 2. Plot the histogram
8 plt.figure(figsize=(8, 5))
9 plt.hist(data_normal, bins=100, density=True, color="salmon", edgecolor="black")
10
11 # 3. Add labels and formatting
12 plt.title("Standard Normal Distribution (np.random.randn)")
13 plt.xlabel("Generated Value Range")
14 plt.ylabel("Density")
15 plt.xlim(-4, 4) # Shows the bulk of the data within 4 standard deviations
16 plt.grid(axis="y", alpha=0.3)
17
18 # 4. Display the plot
19 plt.show()

```



`np.random.randn()`

uses standard normal distribution

values closer to the mean (0) has a higher chance of showing up
usually values show up in the range ± 3 but mathematically
it is unbounded $\pm \infty$

hence it is a bell shape

density is just a rough scaled down version of the amount of
times a value has appeared so the shape looks the same
no matter how many values you sample

jax.grad

- automatic differentiation
- can be mixed with jax.jit

```
installation.ipynb

1 from jax import grad
2
3 def sum_logistic(x):
4     return jnp.sum(1.0 / (1.0 + jnp.exp(-x)))
5
6 x_small = jnp.arange(3.)
7 derivative_fn = grad(sum_logistic)
8 print(derivative_fn(x_small))
```

exp() refers to exponent base e e^x
other num. e.g. 2^2 use $\cdot 2 ** 2$
• `jnp.power(2, 2)`

Line 4: $1.0 / (1.0 + jnp.exp(-x)) = \sigma(x) = \frac{1}{1+e^{-x}}$ }
formula for the sigmoid function (logistic)

what it does?

- takes a real number from $-\infty \leq x \leq +\infty$
- squashes it into a tight range from 0 to 1

what's its purpose?

- turn arbitrary scores to probabilities in ML

properties

- center

if you pass $x=0$ which is the center of the range, it will return 0.5 in the new range.

- positive num.

e.g. put x as a high positive num.

$\therefore e^{-x} \approx \frac{1}{e^x} \approx$ very small number ≈ 0

$\therefore 1 + 0 \approx 1$

$\therefore \frac{1}{1} \approx 1$

- negative num.

e.g. put x as a high negative num

$\therefore e^{-(-x)} \approx e^x \approx$ very big num. $\approx k$

$\therefore \frac{1}{k} \approx 0$

how the cell works?

Step 1.

- defines a sigmoid function
- use `jnp.sum` to combine multiple slopes of tangents to one with different points of contact (x-values) for time-saving purposes

Step 2.

- the `jnp.arange(3)` creates an array of 3 values $[0.0, 1.0, 2.0]$
- substitute into the sigmoid function three times and sum them up

Step 3.

- grad from JAX is very smart
 - it finds out the original function by the summed value
 - it differentiates the function by smart automatic methods from the lib
- and returns an array of 3 values ~~X~~ a summed single value this time since the derivative sum rule is just three individually differentiate values added together. Therefore, returning 3 distinct values in an array.

* Instead of writing a slow loop to calculate the slopes of the three points one at a time. Vectorization (Batch processing) allows JAX to use the computer hardware to calculate all three slopes simultaneously.

new
stuff

```
installation.ipynb

1 from jax import grad
2 import jax.numpy as jnp
3
4 def sum_logistic(x):
5     return jnp.sum(1.0 / (1.0 + jnp.exp(-x)))
6
7 x_small = jnp.arange(3.)
8 derivative_fn = grad(sum_logistic)
9 # print(derivative_fn(x_small))
10
11 def first_finite_differences(f, x, eps=1E-3):
12     return jnp.array([(f(x + eps * v) - f(x - eps * v)) / (2 * eps)
13                       for v in jnp.eye(len(x))])
14
15 print(first_finite_differences(sum_logistic, x_small))
```

Proof of JAX's auto-grad with first finite difference

What is first finite difference?

High school calculus (The Limit Definition)

$$\text{slope} \approx \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

More accurate ver. — Central Finite Difference

$$\text{slope} \approx \frac{f(x + \epsilon) - f(x - \epsilon)}{2\epsilon}$$

$x + \epsilon$: tiny step forward

$x - \epsilon$: tiny step backward

2ϵ : total distance between the two points

What are the arguments?

f : `swm_logistic`

x : `x-small` $\approx [0.0, 1.0, 2.0]$

$\text{eps} = 1\text{E-}3 \approx 0.001$

$\hookrightarrow$ epsilon

What does the for loop in the function do?

for v in `jnp.eye(len(x))`

Creates a 3×3 identity matrix from top-left to bottom right, a diagonal 1.0 is formed

$v1 = [1.0, 0.0, 0.0]$

$v2 = [0.0, 1.0, 0.0]$

$v3 = [0.0, 0.0, 1.0]$

The for loop goes through the function three times,

round 1: $x + \text{eps} * v$

$$= [0.0, 1.0, 2.0] + 0.001 * [1.0, 0.0, 0.0]$$

$$= [0.0, 1.0, 2.0] + [0.001, 0.0, 0.0]$$

$$= [0.001, 1.0, 2.0]$$

$x - \text{eps} * v$

$$= [0.0, 1.0, 2.0] - 0.001 * [1.0, 0.0, 0.0]$$

$$= [-0.001, 1.0, 2.0]$$

Put it into sigmoid function

$$\frac{f(x + e) - f(x - e)}{2e}$$

$$= \left[\frac{1}{1 + e^{-x_0}} + \frac{1}{1 + e^{-x_1}} + \frac{1}{1 + e^{-x_2}} - \left(\frac{1}{1 + e^{-x_0}} + \frac{1}{1 + e^{-x_1}} + \frac{1}{1 + e^{-x_2}} \right) \right] \div 0.002$$

$$= (2.112 \dots - 2.111) \div 0.002$$

$$= 0.2499 \dots \approx \text{auto grad result}$$



installation.ipynb

```
1 print(grad(jit(grad(jit(grad(sum_logistic))))))(1.0))
```

Differentiating a function multiple times can be tedious, with jit it breaks down the complex math into machine code that can be executed by TPU in silicon speed.

However, there's a even better syntax for this,



installation.ipynb

```
1 fast_third_derivative = jit(grad(grad(grad(sum_logistic))))  
2 print(fast_third_derivative(1.0))
```

Which is to put one jit outside of the third derivative.

installation.ipynb

```
1 from jax import jacobian
2 import jax.numpy as jnp
3 x_small = jnp.arange(3.)
4 print(jacobian(jnp.exp)(x_small))
```

Jacobian tracks how every single output responds to every single input

inputs $[x^0, x^1, x^2]$

outputs $[y^0, y^1, y^2]$

$$y^0 = e^{x^0}$$

$$y^1 = e^{x^1}$$

$$y^2 = e^{x^2}$$

★ Take $y^0 = e^{x^0}$ as example, it only responds to how x^0 changes the output. Therefore, other values in the vector are 0

$$\text{Jacobian} = \begin{bmatrix} [1, 0, 0] \\ [0, 2.7182817, 0] \\ [0, 0, 7.389056] \end{bmatrix}$$

★ Jacobian is a matrix filled with first derivatives (slopes), though it may seem like JAX is just storing the value of y , it's actually an illusion of $\frac{d}{dx} e^x = e^x$.

```

1 from jax import jacfwd, jacrev
2 from jax import jit
3 import jax.numpy as jnp
4
5 x_small = jnp.arange(3.)
6 def sum_logistic(x):
7     return jnp.sum(1.0 / (1.0 + jnp.exp(-x)))
8
9 def hessian(fun):
10     return jit(jacfwd(jacrev(fun)))
11 print(hessian(sum_logistic)(x_small))

```

Jacobian is a matrix of first derivatives (slopes)

Hessian is a matrix of second derivatives (concavity of a function)

↓

It tracks how the slope of the function changes as its input changes

`jacrev(fun)`: The inner layer uses reverse-mode differentiation to calculate the first derivative

fewer outputs
than inputs

* reverse-mode is incredibly fast at going from outputs back to inputs

`jacfwd(...)`: The outer layer differentiates the derivative again using forward-mode differentiation

for fewer
inputs than
outputs

* Highly efficient at tracking how those slopes change across your inputs

Therefore, Hessian calculates the second derivative with the fewest possible math operations.

VJP

- give an output change to see how all inputs change
- used by jacrev to build the matrix row by row, from top to bottom

JVP

- give an input change to see how all outputs change
- used by jacfwd to build the matrix column by column, from left to right

Notebook example from last page

$$\begin{aligned} f(x) &= \frac{1}{1+e^{-x}} \\ &= (1+e^{-x})^{-1} \end{aligned}$$

* JAX uses symbolic differentiation rules i.e. Chain rule, quotient rule)

$$\begin{aligned} f'(x) &= -(1+e^{-x})^{-2} (-e^{-x}) \\ &= \frac{e^{-x}}{(1+e^{-x})^2} \\ &= \frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}} \\ &= \frac{1}{1+e^{-x}} \cdot \frac{1+e^{-x}-1}{1+e^{-x}} \\ &= \frac{1}{1+e^{-x}} \cdot \left(1 - \frac{1}{1+e^{-x}}\right) \\ &= f(x) (1 - f(x)) \end{aligned}$$

$$\begin{aligned} f'(x) &= \frac{d}{dx} f(x) \\ &= \frac{d}{dx} f(x) (1 - f(x)) \\ &= f(x) \frac{d}{dx} (1 - f(x)) + (1 - f(x)) \frac{d}{dx} f(x) \\ &= f(x) (-f'(x)) + (1 - f(x)) f'(x) \\ &= f'(x) (-f(x) + 1 - f(x)) \\ &= f'(x) (1 - 2f(x)) \end{aligned}$$

Now put in x -small,

$$f(x)|_{x=0} = 0.5$$

$$|_{x=1} = 0.731058578$$

$$|_{x=2} = 0.880797078$$

$$f'(x)|_{x=0} = 0.25$$

$$|_{x=1} = 0.196611933$$

$$|_{x=2} = 0.104993585$$

$$f''(x)|_{x=0} = 0$$

$$|_{x=1} = -0.090857747$$

$$|_{x=2} = -0.079962501$$

$(0, 0.5)$ is an inflexion point

concavity is + before 0 and - after 0. Therefore, the whole matrix is -ve.

e.g. $x = -1$

$$f(x) = 0.268941921$$

$$f'(x) = 0.196611933$$

$$f''(x) = 0.124282445$$

↪ output of 0.0 or -0.0 depends on JAX's internal floating-point arithmetic but the numerical value is zero.

JAX auto-vectorization

```
installation.ipynb

1 from jax import random
2
3 key = random.key(1701)
4 key1, key2 = random.split(key)
5 mat = random.normal(key1, (150, 100))
6 batched_x = random.normal(key2, (10, 100))
7
8 def apply_matrix(x):
9     return jnp.dot(mat, x)
```

defines a key variable with `random.key` to ensure the random operations are predictable and reproducible

split the key to ensure the values outputted later by random functions are different.

★ if you pass the same key, the output of the random function will be the same

Creates a 2D matrix with a shape 150×100 filled with standard normal distribution values (check prev pages) with key¹

Creates a 2D array of shape 10×100 using key2

↳ same data structure as a matrix, but different purposes in data science

★ `jnp.dot` means to calculate the dot product i.e. matrix multiplication of `mat` and an input `x`.

```

1 def naively_batched_apply_matrix(v_batched):
2     return jnp.stack([apply_matrix(v) for v in v_batched])
3
4 print('Naively batched')
5 %timeit naively_batched_apply_matrix(batched_x).block_until_ready()

```

loops through batched-x row by row (total for 10 rows) for loop
 pulls out one vector (row) of sized 100 at a time
 pass each value into the apply-matrix() function
 which multiplies the size 100 vector with mat ~~as~~ ^{as} ~~matrix~~ ^{matrix} ~~into~~ ^{into} ~~size 100 v matches mat~~ ^{size 100 v matches mat} 150 x 100 and results in a vector of size 150 ^{a column vector}
 loop ten times to create 10 vector of size 150 and stacked together to create a 10x150 matrix

%timeit is a Jupyter Notebook command that runs the function multiple times to calculate the average speed.
 .block-until-ready makes sure time is accurate by forcing Python to wait until GPU math is completed.

Matrix multiplication (note v/e.s.)

capital for matrix

$$A = \begin{bmatrix} 0 & 3 & 5 \\ 5 & 5 & 2 \end{bmatrix} \quad 2 \times 3 \text{ matrix}$$

small for letter

$$w = \begin{bmatrix} 3 \\ 4 \\ 3 \end{bmatrix} \quad \text{also can be a } 3 \times 1 \text{ matrix}$$

= 2x1 matrix

$$\begin{bmatrix} 0 & 3 & 5 \\ 5 & 5 & 2 \end{bmatrix} \begin{bmatrix} 3 \\ 4 \\ 3 \end{bmatrix} = \begin{bmatrix} 0 \cdot 3 + 3 \cdot 4 + 5 \cdot 3 \\ 5 \cdot 3 + 5 \cdot 4 + 2 \cdot 3 \end{bmatrix} = \begin{bmatrix} 27 \\ 41 \end{bmatrix}$$

$[150 \times 100] [100 \times 1]$

= $[150 \times 1]$ matrix

`jnp.dot(mat, v)` turns it into a column vector
↓
* right side

when inside `batched-x` it is a row vector

after processing, JAX stores the 150×1 matrix / column vector into a 70 array

↓
no dimension
ie column/row

after the loop, `jnp.stack` by default creates

a new dimension at the beginning (Axis 0) which represents rows in a 2D matrix. Therefore, becoming 10×150 matrix

```
Installation.ipynb

1 import numpy as np
2
3 @jit
4 def batched_apply_matrix(batched_x):
5     return jnp.dot(batched_x, mat.T)
6
7 np.testing.assert_allclose(naively_batched_apply_matrix(batched_x),
8                             batched_apply_matrix(batched_x), atol=1E-4, rtol=1E-4)
9 print('Manually batched')
10 %timeit batched_apply_matrix(batched_x).block_until_ready()
```

• Python decorator @jit

• passes the function to an XLA compiler

→ previous func. but completed at once

jnp.dot : matrix multiplication of $\text{batched_x} \cdot \text{mat.T}$

$\text{mat} = 150 \times 100$ mat.T flips rows and columns

$\text{batched_x} = 10 \times 100$

$\text{mat.T} = 100 \times 150$

resulting matrix: 10×150

$\text{np.testing.assert_allclose}()$

• safety check

• compares every number from $\text{batched_apply_matrix}$ to $\text{naively_batched_apply_matrix}$

• ensures they match up to $1E-4$ (@.0001)

X or else would throw an error

`np.testing.assert_allclose` checks the difference between two numbers using this formula:

$$\text{Difference} \leq \text{atol} + (\text{rtol} \cdot \text{expected_value})$$

`atol`: absolute tolerance e.g. 0.0001

- protects comparison for numbers very close to zero

`rtol`: relative tolerance

- allows the tolerance level to scale up

e.g. 10000 is the expected-value $10000 \times 0.0001 = 1$

✓ Serves as a silent unit test without a visible output

✗ prints a massive error showing where it didn't work

`jit.dot` uses its built-in batching semantics i.e. Matrix-Matrix Multiplication

Python only talks to the hardware once, instead of a loop 10 separate times which creates a 'launch overhead' time delay.

Parallelism: JAX utilizes all cores from the processor at the exact same time to process the matrix

XLA (Accelerate linear algebra) strips away all the Python and combines operations into hyper-optimized machine code with @jit

manually batched

$56.4 \text{ ms} \pm 15.1 \text{ ms}$ per loop (mean $\pm$ std. dev. of 7 runs, 10000 loops each)

ran the function 70000 times ✱ to calculate the accurate average

$$70000 \times 0.0000504 = 3.528 \text{ s}$$



Installation.ipynb

```
1 from jax import vmap
2
3 @jit
4 def vmap_batched_apply_matrix(batched_x):
5     return vmap(apply_matrix)(batched_x)
6
7 np.testing.assert_allclose(naively_batched_apply_matrix(batched_x),
8                             vmap_batched_apply_matrix(batched_x), atol=1E-4, rtol=1E-4)
9 print('Auto-vectorized with vmap')
10 %timeit vmap_batched_apply_matrix(batched_x).block_until_ready()
```

vmap automatically vectorizes the function
it is slower than manual batching, but less likely to have errors
and you don't have to figure out manual matrix transpose

Pseudorandom Numbers

```
installation.ipynb

1 import numpy as np
2
3 key = np.random.seed(1)
4 random_number = np.random.rand(2)
5 print(random_number)
```

Numpy random number generation relies on a hidden global state which generates the numbers based on it and then updates it behind the scenes. When you try to run this in parallel, multiple processes tries to update the hidden state at the same time which creates race conditions, making the code produces different numbers depending which thread finishes first
X reproducible

```
installation.ipynb

1 from jax import random
2
3 key = random.key(43)
4 print(key)
5 print(random.normal(key, (2,)))
6 print(random.normal(key, (2,)))
```

JAX functions are pure

X global hidden states

✓ same key, always same outputs

★ Splitting a key into 2 subkeys, and input into random.normal functions will create two distinct outputs but no matter how many times it is run, it will have the output

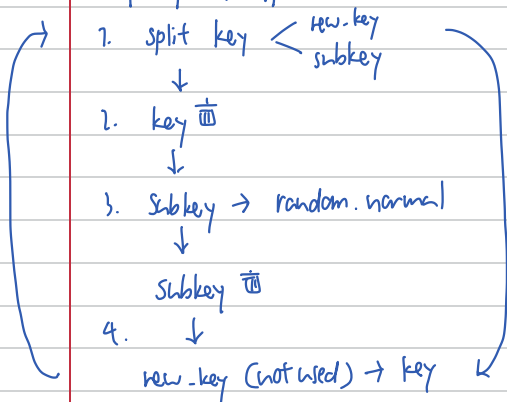
★ splitting the key changes the internal data away of the key

★ Never reuse keys (unless you want identical outputs)

Use `jax.random.split` to split the key before passing it to a random function to generate different, and independent samples.

```
1 from jax import random
2
3 key = random.key(43)
4
5 for i in range(3):
6     new_key, subkey = random.split(key)
7     del key # The old key is consumed by split() -- we must never use it again.
8
9     val = random.normal(subkey)
10    del subkey # The subkey is consumed by normal().
11
12    print(f"draw {i}: {val}")
13    key = new_key # new_key is safe to use in the next iteration.
```

Step-by-step approach



✓ thread safe

✗ race conditions

★ Pseudorandom $\approx$ fake random

- ensures reproducibility
- fast generation speed

Debugging JAX code

For simple inspection: `jax.debug.print`

* Python's `print` only shows results before the runtime values even existed.

Therefore, it only shows tracer values within `jax.jit` decorated code
ie you can't inspect anything meaningful when JIT is involved.

```
1 from jax import jit
2 import jax.numpy as jnp
3
4 @jit
5 def f(x):
6     print("print(x) →", x)
7     y = jnp.sin(x)
8     print("print(y) →", y)
9     return y
10
11 result = f(2.)
```

```
1 import jax
2 import jax.numpy as jnp
3
4 def f(x):
5     print("print(x) →", x)
6     y = jnp.sin(x)
7     print("print(y) →", y)
8     return y
9
10 result = f(2.)
```

JIT Python print result:

`print(x)` → `JitTracer(≈float32[])`
`print(y)` → `JitTracer(≈float32[])`

Without JIT Python print result:

`print(x)` → `2.0`
`print(y)` → `0.9092974`

Use JAX's debug print to display real runtime values

```
1 from jax import jit
2 import jax.numpy as jnp
3
4 @jit
5 def f(x):
6     jax.debug.print("jax.debug.print(x) → {x}", x=x)
7     y = jnp.sin(x)
8     jax.debug.print("jax.debug.print(y) → {y}", y=y)
9     return y
10
11 result = f(2.)
12
13 # print("result →", result)
```

* make sure to include `return y` to
save the result to result.

Or else Python returns nothing despite
doing the calculations, and only saves the
result to the point where `y` is printed.

JIT JAX Debug print result:

`jax.debug.print(x)` → `2.0`

`jax.debug.print(y)` → `0.9092974068739314`

Debugging flags

JAX offers flags and context managers to enable us to catch errors more easily.

e.g. `jax.debug-nans`, `jax-disable-jit`

`jax.debug-nans` $\approx$

NaN = Not a Number ie Math Error

e.g. $\frac{0}{0}$, $\infty - \infty$, $\sqrt{-1}$
 $\downarrow$ real number math

§ In ML/JAX, NaN showing up will turn every calculation that touches it into a NaN.

It spreads like a virus and ruins the training process.

★ stops your code immediately when a NaN pops up

`jax-disable-jit` $\approx$

- toggles off EVERY JIT decorator/function at once globally

- makes the code production ready, and running tests without modifying the entire codebase

Flags and Context Managers

Flags

```
installation.ipynb

1 import jax
2
3 # Flip the global switch to True
4 jax.config.update("jax_disable_jit", True)
5
6 # Now, ANY function with @jit across your whole program
7 # will run as normal, slow, step-by-step Python code.
```

- Flags act as a global set of rules applied to your whole codebase
- You configure them with `jax.config.update`

Context Managers

```
installation.ipynb

1 import jax
2
3 # Global Rule: JIT is active normally.
4
5 # Temporary Context Room: Disable JIT ONLY inside this block
6 with jax.disable_jit():
7     # Inside this room, JIT is temporarily turned off.
8     # You can safely use traditional Python print() or a debugger (pdb) here.
9     result = f(2.)
10
11 # Back to Global Rules: Out of the 'with' block, JIT is automatically turned back on!
```

- Creates a designated space where a specific rule (flag) is toggled on

X global

\$ uses the `with` keyword + flag to set up context manager

\$ `pdb` is a built-in module for debugging that can be executed into commands (`Pdb` is out of this notebook's scope)